

ROS 2 — Lab 3

Your First Python Nodes: Workspaces, Packages, and rclpy

2103581 Robot Operating System and Applications · 2147427 Advanced Mobile Robots

Name: Ben Srinorn

Student ID: 6639112821

Date: 3/9/2026

Objectives

Last week you used other people's nodes. This week you write your own. By the end of this lab you will be able to:

- Create and build a ROS 2 **workspace** with colcon, and explain the underlay/overlay sourcing rule.
- Create a Python package and register executables in setup.py.
- Write a publisher and a subscriber node in Python with rclpy.
- Publish a different message type — geometry_msgs/Twist — and drive the turtle open-loop.
- Close a **feedback loop**: read the turtle's pose and steer it through waypoints.

Prerequisites: a working Lab 2 setup, and the starter files talker.py, listener.py, turtle_circle.py, turtle_writer.py from the course site (lab3_starter_files.zip).

Part 0 — Build tools and your workspace (20 min)

A **workspace** is a folder where your own packages live and get built. Install the build tool and create the workspace:

```
sudo apt install python3-colcon-common-extensions
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws
colcon build
```

Even empty, the build creates three folders. Check with ls: build/, install/, log/ (plus your src/).

0.1 The sourcing rule (read this twice)

Your ~/.bashrc already sources /opt/ros/humble/setup.bash — that is the **underlay**: ROS itself. Your workspace is an **overlay** on top of it. After every build, and in every new terminal where you want to run YOUR packages:

```
source ~/ros2_ws/install/setup.bash
```

Rule of thumb for the whole semester: “*build, then source, then run*”. Ninety percent of today's mysterious errors are a forgotten build or a forgotten source.

CHECKPOINT 1

Show a TA your ~/ros2_ws with the four folders, and run echo \$AMENT_PREFIX_PATH after sourcing.

Q1. Which two locations appear in it, and in which order?

```
bxnchxchi@bxnchxchi-ASUS-TUF-Gaming-F15-FX506LH-FX506LH:~/4sem1/ADV-mo$ ls
build install log src
bxnchxchi@bxnchxchi-ASUS-TUF-Gaming-F15-FX506LH-FX506LH:~/4sem1/ADV-mo$ echo $AMENT_PREFIX_PATH
/home/bxnchxchi/4sem1/ADV-mo/install/package_prefix:/opt/ros/humble
```

Part 1 — Create a Python package (20 min)

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python my_first_pkg \
  --dependencies rclpy std_msgs geometry_msgs turtlesim
```

Explore what was generated (tree my_first_pkg or just ls -R). The two files you will touch today: package.xml (what the package is and what it depends on) and setup.py (how Python installs it, and

which commands it provides). Your code goes in the inner folder `my_first_pkg/my_first_pkg/`, next to `__init__.py`.

Q2. Open package.xml. Which four `<depend>` lines did the create command add?

```
<depend>rclpy</depend>
<depend>std_msgs</depend>
<depend>geometry_msgs</depend>
<depend>turtlesim</depend>
```

Part 2 — The publisher: talker.py (40 min)

Copy `talker.py` from the starter files into `~/ros2_ws/src/my_first_pkg/my_first_pkg/`. This one is complete — we will walk through it together. The core pattern, which every node this semester will follow:

```
class Talker(Node):
    def __init__(self):
        super().__init__('talker')
        self.publisher_ = self.create_publisher(String, 'chatter', 10)
        self.timer = self.create_timer(0.5, self.timer_callback)

    def timer_callback(self):
        msg = String()
        msg.data = f'Hello from Python! count = {self.count}'
        self.publisher_.publish(msg)
```

Now register it as an executable. In `setup.py`, edit `entry_points`:

```
entry_points={
    'console_scripts': [
        'talker = my_first_pkg.talker:main',
    ],
},
```

```
timer = 0.5
average rate: 1.999
min: 0.500s max: 0.501s std dev: 0.00035s window: 3
average rate: 2.000
min: 0.499s max: 0.501s std dev: 0.00050s window: 6
```

Build, source, run:

```
cd ~/ros2_ws && colcon build
source install/setup.bash
ros2 run my_first_pkg talker
```

```
timer = 0.1
average rate: 9.995
min: 0.100s max: 0.101s std dev: 0.00024s window: 12
average rate: 9.997
min: 0.100s max: 0.101s std dev: 0.00023s window: 23
average rate: 9.998
min: 0.100s max: 0.101s std dev: 0.00020s window: 34
```

Verify with last week's tools in a second (sourced!) terminal: `ros2 node list`, `ros2 topic echo /chatter`.

Q3. Change the timer from 0.5 to 0.1, rebuild, and measure with `ros2 topic hz /chatter`. What rate do you get, and why? ~2 to ~10 msg/s because time interval is half

CHECKPOINT 2

Show a TA your talker running via `ros2 run` and its messages appearing in `ros2 topic echo`.

```
[INFO] [1788534784.31527627] [talker]: data: Hello from Python! count = 227
[INFO] [1788534784.41274365] [talker]: data: Hello from Python! count = 228
[INFO] [1788534784.51536791] [talker]: data: Hello from Python! count = 229
[INFO] [1788534784.61499369] [talker]: data: Hello from Python! count = 230
[INFO] [1788534784.71499369] [talker]: data: Hello from Python! count = 231
[INFO] [1788534784.81499369] [talker]: data: Hello from Python! count = 232
```

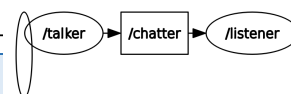
Part 3 — The subscriber: listener.py (30 min)

Your turn. Copy `listener.py` into the same folder and complete its three TODOs — `talker.py` is your reference. Then add a second line to `entry_points` (`'listener = my_first_pkg.listener:main'`), rebuild, and run `talker` and `listener` in two terminals.

Q4. Start a **second** listener in a third terminal. Do both receive the messages about topics? Yes, topic can be subscribed by more than 1 node

```
[INFO] [1788535651.062513576] [listener]: 1 heard: "Hello from Python! count = 1827"
[INFO] [1788535651.162543621] [listener]: 1 heard: "Hello from Python! count = 1828"
[INFO] [1788535651.262543621] [listener]: 1 heard: "Hello from Python! count = 1829"
[INFO] [1788535651.362543621] [listener]: 1 heard: "Hello from Python! count = 1830"
[INFO] [1788535651.462543621] [listener]: 1 heard: "Hello from Python! count = 1831"
```

Q5. Open `rqt_graph`. Draw the graph you see (nodes as circles, topics as arrows).



CHECKPOINT 3

Show a TA your talker + two listeners in `rqt_graph`, and explain the graph in one sentence.

both listeners are same node

Part 4 — A new message type: Twist (20 min)

Strings are for chatting; robots move on `geometry_msgs/Twist` — a linear velocity and an angular velocity. You already inspected it in Lab 2:

```
ros2 interface show geometry_msgs/msg/Twist
```

Here is the point of this part: **nothing else about the pattern changes**. Copy `turtle_circle.py` into the package and compare it with `talker.py` — same Node subclass, same publisher, same timer callback. Complete its three TODOs, add the entry point, rebuild, and run it together with `ros2 run turtlesim turtlesim_node`. The circle values from Lab 2, Checkpoint 4 go straight in.

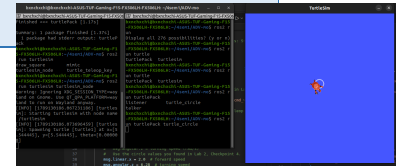
Q6. Exactly which lines differ between `talker.py` and `turtle_circle.py`? (Hint: import, publisher, and what the callback fills in.)

CHECKPOINT 4

Show a TA your turtle driving a circle commanded by YOUR node — no teleop, no topic pub. Bonus: a figure-eight (hint at the bottom of the starter file).

`String` `Twist` `'chatter'` `'turtlesim/cmd_vel'`

```
msg.data = f'Hello from Python! count = {self.count}'  
  
msg.linear.x = 2.0 # forward speed  
msg.angular.z = 6.28 # turning speed
```



Part 5 — Close the loop: turtle calligraphy (40 min)

Your circle node is **open-loop**: it publishes, and hopes — nothing checks where the turtle actually is. A real robot **measures** where it is and corrects itself — a **closed loop**. `turtle_writer.py` does both at once: it subscribes to `/turtle1/pose` and publishes to `/turtle1/cmd_vel`, steering the turtle through a list of waypoints.

5.1 How the controller works

- `pose_callback` just stores the latest pose — the turtle's x, y, and heading θ .
- A 20 Hz timer runs `control_loop`: compute the distance and the heading error to the current waypoint ($\text{math.atan2}(dy, dx)$ gives the direction to the goal), then publish **proportional** commands: turn speed = $K_{\text{ANG}} \times \text{heading_error}$, forward speed = $K_{\text{LIN}} \times \text{dist}$ once roughly facing the goal.
- Always pass heading errors through the provided `normalize_angle()` — an error of $+350^\circ$ must become -10° , or the turtle spins the long way around.

5.2 Build it

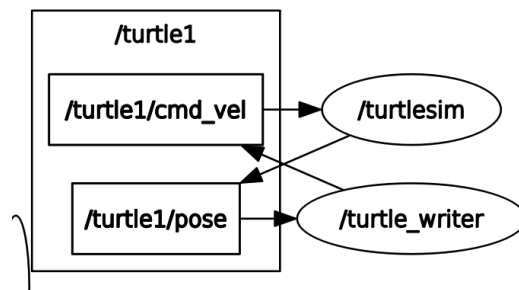
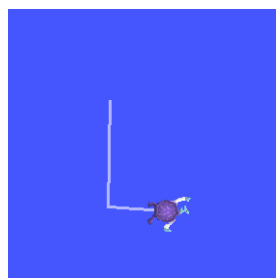
Complete TODOs 1–3 in `turtle_writer.py`, add its entry point, rebuild, and run it together with `ros2 run turtlesim turtlesim_node`. The starter waypoints trace the letter L.

Q7. Raise K_{ANG} from 6 to 20, rebuild, and run again. What does the turtle do near the waypoints, and why? Robot turns faster because /turtle1/cmd_vel → msg.angular & K_ANG

CHECKPOINT 5

Show a TA: (a) your turtle tracing the letter L, and (b) `rqt_graph` — there are now arrows in BOTH directions between your node and `turtlesim`. That circle IS the feedback loop.

Troubleshooting



Symptom	Likely cause and fix
Package 'my_first_pkg' not found	You did not source the overlay in this terminal: <code>source ~/ros2_ws/install/setup.bash</code> — needed in every terminal, after every fresh build.
No executable found	The entry point is missing or misspelled in <code>setup.py</code> — the format is <code>'name = pkg.module:main'</code> . Rebuild after editing.
Code changes have no effect	You must <code>colcon build</code> after every edit — the installed copy is what runs. (Or build once with <code>colcon build --symlink-install</code> so Python edits apply immediately.)
<code>ModuleNotFoundError: rclpy</code>	The underlay is not sourced — open a new terminal or source <code>/opt/ros/humble/setup.bash</code> . Also: never run the file with plain <code>python3 talker.py</code> from an unsourced terminal.
<code>SetuptoolsDeprecationWarning</code> during build	Harmless on Humble — ignore it.
Turtle does not move	Is <code>turtlesim_node</code> running? Is your topic name exactly <code>/turtle1/cmd_vel</code> ? Check with <code>ros2 topic info /turtle1/cmd_vel</code> — it should show 1 publisher, 1 subscriber. In Part 5, also check your pose subscription: no pose received means the controller (correctly) does nothing.
Turtle spins in place or wiggles wildly	Spinning the long way: you skipped <code>normalize_angle()</code> on the heading error. Wiggling/overshooting near waypoints: <code>K_ANGLE</code> too high or <code>GOAL_TOL</code> too small — tune the gains.

Homework (before next week)

1. Modify your talker so it publishes your name and student ID once per second on a topic called `/introduce`. Submit `talker.py` and a screenshot of `ros2 topic echo /introduce`.
2. **Turtle calligraphy (TODO 4):** replace the waypoint list in `turtle_writer.py` so the turtle writes **your name as 3 English initials** — e.g. Surat Kwanmuang → **SKM**. Sketch each letter on graph paper first (about 2 units wide, with a gap between letters); curved letters can be approximated with 4–6 short segments. The initials must be readable in the screenshot. Submit `turtle_writer.py` and a screenshot of the finished drawing. *Your initials are yours alone — identical waypoint lists in two submissions are treated as copying.* Bonus: lift the pen between letters using the `/turtle1/set_pen` and `/turtle1/teleport_absolute` services (hint at the bottom of the starter file).
3. Read ahead: the official Humble tutorial “Writing a simple service and client (Python)” — next week we add services and parameters to our package.

```

x_benchchi@benchchi-ASUS-TUF-Gaming-F15-FX506LH-FX506LH-45em1JA:~$ ros2 topic echo /introduce
data: 'Ben srisoonorn ID: 6638112821'
---
data: 'Ben srisoonorn ID: 6638112821'
---
data: 'Ben srisoonorn ID: 6638112821'
---
data: 'Ben srisoonorn ID: 6638112821'
---
data: 'Ben srisoonorn ID: 6638112821'
---

```



Deliverables: this sheet with Q1–Q7 answered and all five checkpoints signed by a TA by the end of the session; homework 1–2 submitted before next week's class.